

POLICYWATCHER: An Evidence-Preserving Pipeline for Monitoring Terms-of-Service and Privacy-Policy Changes on Consumer Technology Platforms

Fabrizio Degni

PolicyWatcher Project

<https://www.policywatcher.online>

July 2026

Abstract

Terms of service and privacy policies govern the digital lives of billions of people, yet they change silently. Providers routinely amend clauses on data collection, artificial-intelligence training, and content licensing without effective notice to the users bound by them. We present POLICYWATCHER, an open, low-cost platform that continuously acquires the policy pages of major technology and fintech providers, detects substantive changes, and publishes bilingual, plain-language analyses of their impact. The central technical contribution is an *evidence-preserving acquisition pipeline* that keeps public records connected to traceable retrieval evidence and withholds uncertain records. The pipeline combines (i) a five-strategy retrieval cascade that spans direct HTTP, explicit HTTP/2, remote headless rendering, and two freshness-guarded web archives; (ii) a two-layer validation model that separates transport failures from content pathologies such as bot walls, soft not-found pages, and consent interstitials; and (iii) a data-governance layer that encodes the difference between retrieved evidence and development fixtures directly in the schema and suspends publication rather than presenting uncertain data as fact. We specify the freshness guard that prevents an archive snapshot captured before the last live baseline from becoming a new baseline, describe a re-baseline protocol that promotes a dataset from fixtures to evidence without emitting spurious change records or notifications, and report an adversarial self-assessment that identified residual paths through which unverified data could have reached the public timeline, subscriber emails, and search-engine indexes. A dated public operational snapshot of the running platform covers 16 providers and 50 configured documents: 46 records were available through the public-evidence gate, four were suspended pending source verification, and 21 public change-records were visible. The snapshot also documents the contribution of direct retrieval, VPS rendering, and archive paths while distinguishing those operational observations from a benchmark of semantic change-detection or legal interpretation accuracy. The platform runs on commodity shared hosting plus a small virtual private server and is released as open source with continuous integration and third-party quality attestation. We discuss the threat model, the ethics of public-interest scraping, current limitations, and a protocol for longitudinal and expert-annotated evaluation.

1 Introduction

The notice-and-consent model remains the legal backbone of the relationship between consumers and online platforms, despite two decades of evidence that it does not function as notice for ordinary

users. Reading the privacy policies a person encounters in a single year has been estimated to require on the order of two hundred hours [1]. Users overwhelmingly accept terms unread, including clauses that no reasonable person would accept if they were read [2], and the language of the documents themselves is frequently pitched above the reading level of the general public [3, 4]. The problem is compounded by change. Policies are living documents, and the clauses that matter most to citizens, including data retention, cross-border transfer, artificial-intelligence training on user content, and licensing of user-generated works, are precisely the clauses that providers have amended most aggressively during the generative-AI transition. A user who consented in January is bound in June by text they have never seen, and the mechanisms that would surface such a change to the affected population do not exist at scale.

Civil-society initiatives have partly addressed the static version of this problem. ToS;DR crowdsources human-authored summaries and ratings of terms of service [22], and the Electronic Frontier Foundation’s TOSBack project pioneered automated tracking of terms pages [23]. On the research side, a substantial literature has produced annotated corpora [5], million-document longitudinal archives [6], and automated clause-level analysis using natural-language processing and machine learning [9, 10, 14, 11, 12]. Yet a gap persists between *corpus construction*, which is retrospective and research-oriented, and *civic monitoring*, which must be continuous, forward-looking, and communicated in plain language to the people affected.

Closing that gap requires solving an engineering problem that is easy to underestimate. **Reliably obtaining the current, complete text of a policy page is adversarial.** The pages that matter most sit behind bot-mitigation walls, JavaScript-only rendering, geographic content splits, and frequent URL churn. Naïve scrapers silently capture error pages, consent interstitials, or stale cached copies, and every downstream artifact, from a stored diff to a risk score to a subscriber email, inherits that corruption without any signal that it has occurred. The dominant failure mode in this domain is not a crash. It is a plausible wrong value that propagates as if it were fact.

POLICYWATCHER is a running platform that monitors 50 policy documents across 16 technology and fintech providers in up to four jurisdictional variants (European Union, United States, United Kingdom, and Global), detects changes, and publishes bilingual (English and Italian) analyses, risk assessments, comparison matrices, and subscriber notifications. This paper reports its design as a systems contribution, organized around a single question:

How can a public-interest monitoring system keep each public record tied to retrieved evidence, and withhold the record when that condition is not met?

The contributions of this paper are the following.

1. **An evidence-grade acquisition cascade** (Section 4) that combines direct fetching with realistic browser fingerprints, explicit HTTP/2 negotiation for protocol-strict providers, remote headless rendering for script-dependent pages, and two web archives, all governed by a *freshness guard* with a stated timestamp invariant that rejects archive snapshots captured before the latest live baseline (Section 5).
2. **A two-layer validation model** (Section 4.3) that separates transport-level outcomes from content-level pathologies, including bot challenges, soft not-found pages, maintenance pages, paywalls, consent walls, and redirect drift, behind a deterministic result contract that forces every caller to handle failure explicitly, and an egress-side server-side request forgery guard with DNS pinning.
3. **A data-governance layer** (Section 7) that encodes the distinction between fixtures and evidence in the database schema, gates every public surface on evidence status, and defines a

re-baseline protocol for promoting a dataset to production without emitting spurious change events or notifications.

4. **A threat model and an adversarial self-assessment** (Sections 8 and 9) conducted against the running system, framed by two questions: where can uncertain data still become public evidence, and which function can destroy or promote data irreversibly if an operational state is wrong. We report the findings and the guardrails that closed them.
5. **An implementation and reproducibility account** (Section 10) covering the open-source release, the automated test suite, continuous integration, and third-party quality attestation, together with a deployment blueprint (Section 12) that runs the entire platform on commodity shared hosting plus one small virtual private server.

2 Background and Related Work

2.1 The readability and change problem

The mismatch between the legal function of policies and their practical readability is well documented. McDonald and Cranor quantified the aggregate time cost of reading policies [1]; Reidenberg et al. demonstrated systematic mismatches between policy meaning and user understanding [4]; and large-scale readability analyses confirm that policies routinely exceed the comprehension level of the general population [3]. Obar and Oeldorf-Hirsch showed experimentally that users accept unread terms at near-universal rates [2]. These results motivate a monitoring system oriented toward the affected public rather than toward researchers, and they motivate a focus on *change*, because a reader who cannot be expected to read a policy once certainly cannot be expected to re-read it after every amendment.

2.2 Policy corpora and longitudinal analysis

Wilson et al. introduced OPP-115, the first richly annotated privacy-policy corpus, establishing a data-practice taxonomy that later systems reused [5]. Amos et al. curated more than one million policy snapshots spanning two decades by mining the Wayback Machine, and documented both the growth of policy length over time and the methodological hazards of archive-based curation, notably error pages and boilerplate masquerading as policies [6]. Linden et al. analyzed the policy landscape before and after the General Data Protection Regulation and found measurable changes in length and disclosure [7]. The content validation layer in POLICYWATCHER (Section 4.3) operationalizes the hazards these corpora identified as run-time checks, and the freshness guard (Section 5) addresses a failure mode specific to *live* monitoring that retrospective corpora, which fix a snapshot set in advance, do not face.

2.3 Automated policy and terms analysis

A rich line of work applies natural-language processing to policies and terms. Privee and Polisris demonstrated automated classification and question answering over privacy policies [8, 9]; PrivaSeer scaled corpus construction to the open web [10]; PolicyLint and PoliGraph extracted and reasoned over data-flow statements, including internal contradictions [11, 12]; and MAPS and related systems scaled compliance analysis to mobile-app ecosystems [13]. In the terms-of-service setting specifically, the CLAUDETTE line of work trained detectors of potentially unfair contractual clauses [14]. POLICYWATCHER consumes a general-purpose large language model for bilingual summarization

and risk annotation, but treats the model as an *untrusted component*: its outputs are schema-validated, range-checked, and, by policy, never a substitute for retrieval evidence (Section 6). The system’s contribution is therefore complementary to this literature. It is not a new clause classifier but an infrastructure that supplies the classifier, whatever it is, with a validated retrieval path and publishes its outputs only when the provenance condition is met.

2.4 Consent interfaces and web measurement

A parallel measurement literature studies the interfaces that gate access to policy-governed services. Studies of cookie consent notices and their dark patterns [15, 16, 17, 18] document precisely the consent walls and interstitials that a policy scraper must detect and reject rather than ingest. Work on realistic and reproducible web crawling [20] and on the detection of browser fingerprinting [21] frames the technical arms race that makes reliable retrieval adversarial and that motivates the multi-strategy cascade in Section 4. Automated auditing of third-party disclosure in policies [19] further underscores that a policy’s text and a site’s actual behavior can diverge, which is one reason POLICYWATCHER anchors every published record to a specific retrieval event.

2.5 Monitoring services and positioning

TOSBack [23] and ToS;DR [22] are the closest antecedents in spirit. TOSBack tracks raw page text and TOSBack-style diffs; ToS;DR provides human-curated ratings. POLICYWATCHER differs in three respects. First, it invests in adversarial retrieval, which is required because modern policy pages actively resist naive fetching in ways that were less prevalent when earlier trackers were designed. Second, it maintains a per-document provenance model in which every published datum links to a check log that records the transport used, the HTTP status, the content hash, and the source of retrieval. Third, it governs the boundary between demonstration data and evidence explicitly, which we found in practice to be the dominant integrity risk (Section 9). Regulatory context is provided by the transparency obligations of the General Data Protection Regulation [26] and the terms-and-conditions provisions of the Digital Services Act [27]; independent monitoring gives such obligations empirical traction.

3 System Overview

POLICYWATCHER is implemented as a server-rendered web application (Next.js with the App Router, React, and TypeScript) backed by a relational database accessed through an object-relational mapper (Prisma over SQLite). Policy analysis calls a hosted large language model. Headless rendering, which requires a Chromium runtime that shared hosting cannot provide, runs as a separate authenticated service (Section 12). The platform comprises four subsystems.

1. **Acquisition:** a scheduled scanner that, for each configured policy URL, executes the retrieval cascade and validation of Sections 4 and 4.3, producing either a normalized text with a SHA-256 content hash or an explicit failure status.
2. **Change detection and analysis:** hash comparison against the stored baseline, and, on change, a stored full-text line diff plus a model-generated analysis comprising bilingual summaries, key points, remediation guidance, a risk score, a fifteen-indicator assessment, and per-region impact statements (Section 6).

3. **Publication:** a public dashboard, a change timeline, per-change pages with diffs, a cross-company comparison matrix over the indicator taxonomy, an evidence-ranked signals board, a regulatory observatory, embeddable widgets, downloadable reports, and subscriber email digests, all gated on evidence status (Section 7).
4. **Operations and audit:** an administrative surface exposing per-policy check logs, dataset-quality findings with severity-ranked blockers, an append-only review log, an access log, and health and control endpoints for the remote renderer.

Data model. Persistence centers on ten entities. **Company** and **Policy** hold the configured monitoring scope; a policy carries its source URL, jurisdiction, current text and hash, a lifecycle value, and the ingestion method that records how the current text was obtained. A policy snapshot stores versioned full texts, each flagged as public evidence. A policy-change record stores the diff and the analysis fields. A policy-check log stores one row per retrieval attempt, recording status, source, HTTP status, final URL, content hash, text length, failure reason, and, for archive retrievals, the snapshot timestamp. Three further entities provide dataset-QA issue review, administrative review, and access logging; region-impact and subscriber records complete the model. The check log is the provenance spine of the system: public records retain a link to the attempt that produced their source evidence.

4 The Evidence Acquisition Cascade

4.1 Design invariant: evidence or explicit status

The pipeline’s contract is that a retrieval attempt returns exactly one of three statuses. **ok** carries validated text and a hash. **unavailable** signals a transient failure such as a timeout, a bot block, or a maintenance page. **invalid** signals that the URL no longer denotes the intended document, for example a hard not-found response, a soft not-found page, or a redirect that drifts to an unrelated destination. Nothing else is representable in the result type. A caller cannot receive partial text without a hash, or a hash without provenance. Failure paths write a check-log row and a user-facing notice that directs the reader to the official source, rather than an approximation. This shape was chosen after observing that most data-integrity incidents in scraping systems are not exceptions but plausible wrong values that propagate silently.

4.2 The retrieval cascade

Strategies are ordered by fidelity to the live site. Archives come last because they can only witness the past. Table 1 summarizes the cascade.

Direct fetch. The primary path issues an HTTP/1.1 request with a realistic desktop browser fingerprint (User-Agent, client-hint headers, and fetch-metadata headers), follows redirects manually with server-side request forgery re-validation at every hop, and retries on status 429 and 5xx with exponential backoff. The request timeout of twenty seconds covers the response *body*, not merely the response headers, which matters against slow-drip responses that deliver headers promptly and then stall. Responses whose content type is clearly not HTML, such as PDF, images, or octet streams, are rejected rather than parsed. The transport makes up to three attempts and rotates among three browser profiles.

Explicit HTTP/2. Several large providers reject HTTP/1.1 requests from non-browser clients with protocol-level 400 responses. A dedicated client negotiates HTTP/2 over ALPN directly.

Order	Strategy	Role and rationale
1	Direct HTTP/1.1	Realistic desktop browser fingerprint; manual redirects with per-hop validation; body-aware timeout. Primary path.
2	Explicit HTTP/2	Direct ALPN negotiation for providers that reject HTTP/1.1 from non-browser clients with protocol-level errors.
3	Rendered fetch	Remote headless Chromium executes JavaScript; recovers single-page-application shells and many bot-protected pages. Optional.
4	Wayback Machine	Freshness-guarded archived snapshot; witness of last resort.
5	Common Crawl	Freshness-guarded archive; gzip-compressed records are decompressed before extraction.

Table 1: The five-strategy retrieval cascade. Strategies 1 to 3 observe the live site; strategies 4 and 5 are archives subject to the freshness guard of Section 5.

Response chunks are accumulated as bytes and decoded once, because decoding at arbitrary chunk boundaries corrupts multi-byte UTF-8 sequences, and redirects are treated as failures at this layer by construction.

Rendered fetch. Pages that ship as JavaScript application shells, which is increasingly the norm for exactly the providers most relevant to consumers, are delegated to a self-hosted rendering service (Section 12). The service loads the page in headless Chromium, waits for hydration and for network quiescence, and returns the rendered document. If the service is not configured or is unreachable, the cascade continues to the archives. Rendering is an amplifier, not a dependency.

Freshness-guarded archives. The Wayback Machine [24] and Common Crawl [25] serve as witnesses of last resort. Common Crawl records are located by CDX lookup and fetched by byte-range reads of gzip-compressed WARC records, which are decompressed before parsing. Both archive strategies enforce the freshness guard of Section 5.

4.3 Content validation

The second layer validates *what* was retrieved. Its checks are pure functions of the retrieved bytes and are unit-tested in isolation.

- **Block-page detection:** signatures of CAPTCHA and JavaScript challenges, web application firewall denial pages, maintenance pages, paywalls and login walls, and consent interstitials, matched only against the document head so that a policy which legitimately *discusses* CAPTCHAs is not itself flagged.
- **Soft not-found detection:** short pages using not-found phrasings in multiple languages, which catches servers that return status 200 for missing documents.
- **Structural extraction:** removal of boilerplate (navigation, headers, footers, cookie banners, hidden elements) and conversion of the main content region to deterministic, structured plain text. The *same* normalizer feeds storage, hashing, and diffing, so cosmetic markup changes do not register as policy changes. When a configured URL carries a fragment identifier, extraction is scoped to the addressed section.
- **Plausibility checks:** a minimum length of 800 characters and a policy-marker vocabulary test that requires at least three of twenty domain terms, which rejects well-formed pages that are not policies. Extracted text is capped at 500,000 characters to bound storage.

- **Drift rejection:** a live retrieval whose final URL, after redirects, lands on a different registrable domain (host drift) or on a site root when a document path was configured (path drift) is rejected as `invalid` rather than ingested. This closes the failure path in which a redirect to a homepage whose footer contains the word “privacy” would otherwise pass the marker test.

4.4 Egress protection

Because the scanner fetches administrator-configured URLs from server-side infrastructure, every fetch passes a server-side request forgery guard [28]: a scheme allowlist, rejection of credentials embedded in the URL, DNS resolution with blocking of private, loopback, and link-local addresses, and re-validation on every redirect hop. To close the time-of-check-to-time-of-use gap between validation and connection, the HTTP/1.1 and HTTP/2 clients pin the validated address at connection time rather than re-resolving the hostname, and the IPv6 address parser is hardened against compressed and mixed notations. The companion renderer applies the same address policy to every browser-initiated subresource request and checks redirect coherence against the Public Suffix List.

5 The Freshness Guard

The archive strategies introduce a subtle integrity risk that is specific to live monitoring. Suppose the live site is temporarily blocked. A naive cascade would fall through to an archive and recover an older copy of the document, whose hash differs from the stored baseline. The system would then register a *change*, run analysis, and notify subscribers about a regression to a superseded text. When the live site later recovers, a second spurious change fires. We eliminate this oscillation class with a single rule.

An archived snapshot is admissible only if its capture timestamp is not earlier than the policy’s last successful live retrieval.

Let t_ℓ be the timestamp of the last retrieval that produced a stored baseline, and let t_s be the capture timestamp of a candidate archive snapshot. The guard admits the snapshot only if $t_s \geq t_\ell$.

Invariant 1. *Provided that the recorded archive capture timestamp is accurate, the freshness guard rejects any candidate snapshot captured before the current baseline.*

Proof sketch. The stored baseline was written by a retrieval at time t_ℓ , and by the pipeline invariant it reflects the document observed at t_ℓ . An archive snapshot is admitted only when $t_s \geq t_\ell$, so any admitted snapshot was captured no earlier than the current baseline. A change is recorded only when an admitted retrieval yields a hash different from the baseline hash. Therefore a recorded change from an archive source refers to a capture at or after t_ℓ , not before it. The rule prevents a pre- t_ℓ capture from becoming a new baseline under those assumptions. Snapshots lacking a timestamp are treated as inadmissible, so the rule does not rely on an unknown capture time. $\square$

The guard is enforced in both archive strategies. Admitted archive retrievals additionally carry their capture timestamp into the check log, so that the age of recovered evidence is recorded and can be displayed to readers. A practical side effect is a reduction in load on public-interest archive infrastructure, since stale snapshots are rejected before they are fetched in full.

Group (5 indicators each)	Indicators and representative values
Privacy and data protection	Data collection (Minimal / Moderate / Extensive); Third-party sharing (Restricted / Limited / Broad); Data retention (Defined / Extended / Indefinite); Right to deletion (Full / Partial / Not available); Cross-border transfer (Restricted / Controlled / Unrestricted).
AI governance	Training opt-out (Available / Opt-out / Not available); Output ownership (User retained / Shared / Company retained); Algorithmic transparency (Published / Mentioned / Opaque); Automated decisions (Transparent / Partial / Opaque); Bias and fairness (Committed / Mentioned / Absent).
Ethics and corporate governance	Consent mechanism (Explicit opt-in / Opt-out / Implicit); Regulatory compliance (Comprehensive / Partial / Minimal); Breach notification (Within 24h / Within 72h / Unspecified); Independent audit (Certified / Mentioned / Absent); Content moderation (Transparent / Partial / Opaque).

Table 2: The fifteen-indicator taxonomy used for cross-company comparison. Every indicator additionally admits the value “Not assessed”, which is the default until an assessment is made.

6 Change Detection and Analysis

Change detection is hash equality over normalized text. On inequality, the system stores a new immutable snapshot and a full-text line diff. An earlier design truncated the diff to the first ten thousand characters, which silently hid changes beyond the opening pages of long documents; the current design stores the complete diff. The system then generates an analysis with a hosted large language model. The primary model is **gemini-2.5-flash**; on transient errors such as rate limiting or temporary unavailability, the system falls back to **gemini-2.0-flash-lite**. Generation uses a low temperature and requests a JSON response.

The analysis schema contains bilingual executive summaries, a one-sentence take-away in each language, three to five key points with sentiment, exactly three risk rationales, an overall risk band, an integer risk score from 1 to 10, two to four remediation actions with links to official opt-out or objection mechanisms, exactly six per-region impact statements (the cross product of three regions and two perspectives), and the fifteen-indicator assessment of Table 2.

Three properties keep the model subordinate to evidence. First, analyses are generated only for retrieved text; there is no path from model output to policy text. Second, model outputs are parsed against a strict schema with range validation: an out-of-range risk score or an unrecognized risk band causes the analysis to be rejected rather than clamped, and indicator values are passed through a case-insensitive whitelist that maps anything unrecognized to “Not assessed”. Third, in production the system prefers to block ingestion of an analysis over substituting a heuristic mock; the demonstration fallback is available only when an explicit environment flag is set. The conversational interface that answers user questions over a policy applies an explicit prompt-injection guard that treats the policy context as untrusted input and returns an honest “unavailable” response rather than fabricating an answer.

7 Data Governance: Fixtures versus Evidence

A monitoring platform needs demonstration data before its first production scan, and it needs development fixtures forever after. The dominant integrity risk we encountered in practice was not

a scraping failure but *fixture leakage*: hand-authored demonstration records, complete with plausible dates and risk scores, reaching public surfaces indistinguishably from evidence. POLICYWATCHER addresses this structurally.

Schema-level distinction. Every policy carries an `ingestionMethod`, which is either `Seeded` or one of the retrieval sources (`direct`, `http2`, `rendered`, `wayback`, `commoncrawl`), and a `dataStatus` lifecycle value drawn from `Configured`, `Available`, `Partial`, `Needs Review`, `Unavailable`, and `Reviewed`. Snapshots and changes additionally carry a `publicEvidence` boolean. Fixtures are born `Seeded` and `Configured`; they are not treated as retrieved evidence.

Public data gate. Every public surface, which includes the read APIs, the server-rendered pages, the share and embed routes, the OpenGraph image routes, the downloadable reports, the sitemap, and, critically, the subscriber emails, filters on evidence status. To be public, a policy must have a status in the set `{Available, Reviewed}`, must not be `Seeded`, and must have at least one snapshot flagged as public evidence; changes must themselves be flagged as public evidence. Policies in a suspended state are represented publicly only by a minimal “source under verification” notice, with no text, no scores, and no timeline events. The gate is a single shared predicate applied uniformly; Section 9 explains why centralization matters.

A consequence of this design, which we state plainly because it is unusual, is that the platform’s bundled demonstration dataset of 16 companies and 50 policy documents is currently suspended in its entirety from public view. Because every seeded record is `Seeded` and `Configured`, the public gate admits zero of the 50 policies and zero of the seeded changes. The demonstration data is useful for development and for exercising the user interface, but the governance layer refuses to present it as evidence. The platform is, by construction, publicly silent about any document it has not actually retrieved.

Re-baseline protocol. Promoting the dataset from fixtures to evidence poses a trap. The first real retrieval of each policy necessarily differs from its placeholder, and a naive pipeline would emit a wave of fabricated “changes”, each with a risk score and an email. The re-baseline path detects seeded provenance, replaces the placeholder with the retrieved text as a fresh first version, removes the fixture snapshots and changes, preserves audit-log entries by detaching rather than deleting them, and emits no change record and no notification. The operation is logged with a distinct reason code. Crucially, the predicate that authorizes this destructive operation is not a single mutable flag: a policy is eligible for re-baseline only if its ingestion method is seeded *and* it has no verified source evidence in its check log *and* it has no snapshot marked as public evidence. History is destroyed only when the evidence itself shows there is no real history to lose.

Quality gates as release blockers. A dataset-assurance tool audits the database for source-evidence defects and treats them as blockers rather than warnings, because the intended production state is to remain publicly silent until evidence exists. The command-line tool uses two severities, blocker and warning, and fails the build if any blocker is present. The administrative console applies a richer three-level scheme (critical, warning, informational) across roughly twenty issue areas, among them Source Fit, which flags a global policy whose URL uses a regional path segment or an EU-or-US policy pointing at a locale-specific page; Coverage, which flags missing or very short text and missing analysis; Freshness, which flags policies not successfully checked in more than thirty days; and Hash Integrity, Snapshot Chain, and Public Evidence Gate checks. Each

finding carries a stable identifier and is reconciled against an append-only review table, so that a human decision to accept or ignore a finding is itself recorded with its author and rationale.

8 Threat Model

We consider three classes of adversary and one class of operator error. The first adversary is the *monitored provider* that, deliberately or not, serves bot walls, geo-splits, or JavaScript-only shells that frustrate retrieval; the cascade and validation layers are the response. The second is the *network position* that could induce the server-side fetcher to reach internal infrastructure; the server-side request forgery guard with DNS pinning is the response. The third is the *content adversary* that embeds instructions in policy text intended to steer the language model; the analysis layer treats retrieved text as untrusted and validates model outputs against a schema. The fourth class, which our self-assessment found to be the most consequential in practice, is *operator error*: a wrong status, a mis-scoped deletion, or a deployment artifact that overwrites live data. The governance layer, the evidence-based re-baseline predicate, and the operational guardrails of Section 9 address this class. We explicitly do not attempt to cover a compromised hosting environment, a leaked administrative secret used within its validity window, or an archive that itself serves corrupted data without an observable indication.

9 Adversarial Self-Assessment

Before the first production re-baseline, we conducted a structured red-team review of the running system, organized around two questions:

1. Where can uncertain, seeded, incomplete, or badly-retrieved data still become public evidence, a notification, a timeline entry, a risk score, or apparently reliable history?
2. Which function can delete, overwrite, or promote data irreversibly if an operational state is wrong?

The review combined manual code audit, live probing of external dependencies, database inspection, and parallel automated code exploration. Table 3 summarizes representative confirmed findings, all subsequently closed. We report the methodology and the failure classes rather than dwelling on individual defects, because the two guiding questions generalize to any civic-evidence platform and, in our experience, surfaced cross-cutting risks that component-level checklists did not.

10 Implementation, Testing, and Reproducibility

POLICYWATCHER is released as open source. The retrieval and validation logic is factored so that its decision functions, including URL and address validation, block-page and soft not-found detection, text extraction, the freshness guard, and the drift checks, are pure and unit-tested; the public-data gate and the governance predicates are likewise unit-tested. At the time of writing the suite comprises 89 tests across 12 modules and passes in full. Network-dependent paths, which are not amenable to deterministic unit testing, are exercised by integration smoke tests against live sources. Continuous integration runs five workflows on each change: a quality gate (schema validation, build, lint, dependency audit, and the dataset-assurance check), a coverage workflow that

Class	Finding	Guardrail
Ungated egress	Periodic email digests queried changes by recency alone, bypassing the gate that protected web surfaces, so fixture changes could have been mailed to subscribers as real intelligence.	Apply the same evidence predicate to every egress channel, including email, feeds, and the sitemap; a time window is not a filter.
Enumeration	The sitemap enumerated all change identifiers, including gated ones, advertising non-public resources to crawlers.	Gate the sitemap on the same predicate as the pages it references.
Archive regression	A transient block on a live site could reinstate an outdated archived copy as a change; a legacy cache fallback had also become dead weight upstream.	The freshness guard of Section 5; removal of the retired fallback.
Irreversible operation	Re-baseline, cascade deletion of a company, and the inclusion of a database file in a deployment artifact each offered a single-step path to irreversible loss under operator error.	Gate destruction on evidence, not on a flag; exclude data stores from deployment artifacts; separate the session-signing secret from the API secret; require an encrypted export before promotion.

Table 3: Representative findings from the adversarial self-assessment and the guardrails that closed them.

uploads to a coverage service, static analysis with CodeQL, a SonarCloud scan, and an OpenSSF Scorecard run. The project additionally holds an OpenSSF Best Practices attestation. We stress that these are engineering-quality signals, not legal or compliance certifications, and the platform’s public documentation states this explicitly.

We report these facts as implementation context, not as an assessment of legal, semantic, or model-analysis correctness. The per-attempt check-log schema was designed so that the evaluation of Section 11 can be reconstructed from operational data.

11 Operational Evaluation of the Running Platform

This section reports a reproducible, point-in-time operational measurement of the running public platform. It deliberately answers narrower questions than a semantic benchmark: what fraction of the configured inventory is exposed through the public-evidence gate; which retrieval path is recorded for the current inventory; and whether a seeded fixture instance exposes its placeholder records. It does *not* measure the correctness of a detected policy change, the correctness of legal or model-generated interpretation, or latency across repeated runs.

11.1 Protocol and data artifact

The production snapshot was collected at 2026-07-11 21:33:25 UTC from four unauthenticated JSON endpoints of the running platform: `/api/leaderboard`, `/api/source-suspensions`, `/api/changes?page=1&pageSize=50`, and `/api/companies`. The collection capture script and the resulting JSON record are included with the paper source. They access no administrative API, credentials, subscriber information, private check logs, or raw policy text. A local development

Public operational measure	Production	Seeded control	Interpretation
Configured policy records	50	50	Same monitored inventory cardinality.
Public-evidence records	46 (92.0%)	0 (0.0%)	Records admitted to the public data gate.
Suspended records	4 (8.0%)	50 of 50	Minimal notices only; no policy text or analysis is exposed.
Publicly listed providers	15 of 16	0 of 16	Provider list after public-data filtering.
Public change-record inventory	21	0	Inventory of records that passed the public change gate, not a semantic-accuracy score.

Table 4: Dated operational snapshot from the production public APIs and a local seeded-fixture control. The two instances have the same configured inventory but different evidence states.

instance with the seeded fixture inventory was queried with the same endpoints as a control for the public-data gate.

The data artifact distinguishes a currently available record from a record that is merely configured. It records the last retrieval path in the public leaderboard, not a repeated-trial success probability. In particular, a **PolicyChange** record visible through the public API is counted as a public change-record inventory item; this evaluation does not independently validate that record as a provider’s legal amendment.

11.2 Results

Table 4 reports the public production state. Of the 50 configured documents, 46 (92.0%) satisfied the public-evidence gate and four (8.0%) were suspended. Fifteen of 16 configured providers had at least one publicly listed record. All four suspended documents belonged to one provider and covered its EU and UK privacy-policy and terms-of-use variants; their public notice exposed the suspension state but no policy text, score, timeline item, or analysis.

The retrieval-path inventory contains 40 direct, five VPS-rendered, two archive-labeled, and three records without a current retrieval path. Of the 46 public-evidence records, 40 were direct, five were VPS-rendered, and one had an archive path. The other archive-labeled record and the three records without a current path belong to the suspended group, which remains withheld from public policy content. The renderer is therefore a recorded path for five currently public records in this snapshot, rather than a theoretical fallback.

The seeded-fixture control produced zero public-evidence records, zero public change records, and zero publicly listed providers while returning 50 suspension notices. This contrast is evidence that the tested public endpoints respect the fixture-versus-evidence boundary under the supplied fixture state. It is not a complete proof that every present or future output channel is free of leakage; the code-level public-data gate and its test suite provide the complementary engineering evidence described in Section 10.

11.3 What the snapshot does not establish

The measurement has no independently annotated corpus of known policy revisions, no expert legal coding, no repeated scan window, and no controlled baseline system. It therefore cannot estimate precision or recall for change detection, agreement with legal experts, robustness over time, or comparative advantage over other monitoring systems. The 21 public change records are useful for

inventory and interface testing, but should not be read as 21 independently verified legal events. A rigorous next study requires a versioned corpus of provider-published texts, timestamped ground truth, blinded expert annotation, and a multi-week repeated-scan protocol.

12 Deployment and Operations

A design constraint throughout was operability by a single individual at consumer cost. The web application, comprising the dashboard, the APIs, and the scanner, runs on commodity shared hosting without root access. Headless rendering, which requires a Chromium runtime, runs as an authenticated service of a few hundred lines on a small virtual private server, reached over HTTPS and treated as optional at every call site. The renderer refuses to start without its bearer secret, applies the same address policy as the main application to every subresource, blocks heavy subresources such as images and fonts for speed, uses a fresh browser context per request to avoid cookie bleed, and bounds concurrency. A separate operations agent on the same server performs backup, health probing, update, and rollback. Scans are batchable, with bounded batch sizes and per-company selection, to respect shared-hosting execution limits, and per-domain politeness delays together with retry backoff bound the request rate to any single provider. The production database is located outside the extracted application directory so that redeploying the application cannot overwrite live data, a safeguard added in direct response to the self-assessment of Section 9. The resulting monthly operating cost is on the order of a few tens of euros, which we consider part of the contribution: longitudinal policy accountability does not require institutional infrastructure.

13 Ethical Considerations

Purpose and proportionality. The system retrieves only public legal documents that providers are already obligated to make available to their users. It does so at low frequency, on the order of weekly scans of 50 documents, with politeness delays. It collects no personal data and exercises no platform functionality beyond reading published pages.

The fingerprinting tension. Reliable retrieval currently requires browser-realistic request fingerprints, because several providers block identified crawlers from the very pages that bind their users. That fact is itself a transparency finding. We document the tension rather than hide it, and the roadmap includes an honest-first mode that first attempts retrieval with an identifying agent and a contact URL and escalates to browser-realistic fingerprints only when refused, together with the publication of per-provider accessibility statistics.

Archives and load. Archive fallbacks impose load on public-interest infrastructure. The freshness guard has the side effect of minimizing archive queries, and archive-derived data is labeled with its capture time so that readers can judge its currency.

Publication integrity. The governance layer reflects an ethical stance. For a platform whose purpose is trust, publishing nothing is strictly better than publishing plausible fabrication. Every user-facing analysis carries its provenance and links to the official document, and the analysis layer is presented to users as AI-assisted rather than authoritative.

14 Limitations and Future Work

Evaluation. Section 11 reports a dated operational snapshot and a seeded-fixture control. It does not report retrieval success rates *over time*, false-change rates, or agreement between model-generated risk annotations and expert legal coding. The check-log schema can support the first two measures; the next study should pre-register a time window, preserve a provider-versioned reference corpus, and include blinded expert review of a sampled set of detected and non-detected changes.

Long-document analysis. Truncation before model analysis can miss clauses deep in long documents. We are implementing a chunked map-reduce pipeline with an explicit coverage manifest that records which spans of the document each conclusion derives from, enabling audits for missed clauses and for conclusions unsupported by the text.

Raw evidence retention. The system stores normalized text. Retaining compressed raw HTML per snapshot, with retention tiers, would strengthen the evidentiary chain and permit retrospective re-extraction as the normalizer improves.

Extraction robustness. Accordion widgets, tabbed legal hubs, and multi-document pages remain extraction hazards. Fragment-scoped extraction mitigates but does not eliminate them, and a per-provider extraction test corpus with reference outputs is in progress.

Coverage and scale. Fifty documents across sixteen providers is a deliberate depth-over-breadth choice. Scaling to hundreds of providers requires per-domain scheduling, conditional requests using entity tags and modification times, and automated URL rediscovery that follows legal-hub links and sitemaps when a configured URL dies.

15 Conclusion

POLICYWATCHER shows how continuous monitoring of the legal documents that govern consumer technology can be organized around retrieval provenance and explicit withholding states at civic cost. The mechanisms reported here, namely the freshness-guarded retrieval cascade, the deterministic failure contract, the fixture-versus-evidence boundary enforced at public egress, the evidence-gated re-baseline protocol, and the adversarial questions that structure the self-assessment, are reusable design patterns for systems that turn scraped web content into public records. The operational snapshot is intentionally modest: it records what the running platform exposed and withheld on a specific date, while leaving semantic and legal-interpretation accuracy to a future expert-annotated, longitudinal study.

Availability

The platform is publicly accessible at <https://www.policywatcher.online>. The source code is released as open source and is subject to continuous integration and third-party quality attestation as described in Section 10.

References

- [1] A. M. McDonald and L. F. Cranor. The cost of reading privacy policies. *I/S: A Journal of Law and Policy for the Information Society*, 4(3):543-568, 2008.
- [2] J. A. Obar and A. Oeldorf-Hirsch. The biggest lie on the internet: ignoring the privacy policies and terms of service policies of social networking services. *Information, Communication and Society*, 23(1):128-147, 2020.
- [3] B. Fabian, T. Ermakova, and T. Lentz. Large-scale readability analysis of privacy policies. In *Proc. Int. Conf. on Web Intelligence (WI)*, 2017.
- [4] J. R. Reidenberg, T. Breaux, L. F. Cranor, B. French, et al. Disagreeable privacy policies: mismatches between meaning and users’ understanding. *Berkeley Technology Law Journal*, 30(1):39-88, 2015.
- [5] S. Wilson, F. Schaub, A. A. Dara, F. Liu, S. Cherivirala, P. G. Leon, et al. The creation and analysis of a website privacy policy corpus. In *Proc. 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2016.
- [6] R. Amos, G. Acar, E. Lucherini, M. Kshirsagar, A. Narayanan, and J. Mayer. Privacy policies over time: curation and analysis of a million-document dataset. In *Proc. The Web Conference (WWW)*, 2021.
- [7] T. Linden, R. Khandelwal, H. Harkous, and K. Fawaz. The privacy policy landscape after the GDPR. *Proceedings on Privacy Enhancing Technologies (PoPETs)*, 2020(1):47-64, 2020.
- [8] S. Zimmeck and S. M. Bellovin. Privee: an architecture for automatically analyzing web privacy policies. In *Proc. 23rd USENIX Security Symposium*, 2014.
- [9] H. Harkous, K. Fawaz, R. Lebrete, F. Schaub, K. G. Shin, and K. Aberer. Polisis: automated analysis and presentation of privacy policies using deep learning. In *Proc. 27th USENIX Security Symposium*, 2018.
- [10] M. Srinath, S. Wilson, and C. L. Giles. Privacy at scale: introducing the PrivaSeer corpus of web privacy policies. In *Proc. 59th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2021.
- [11] B. Andow, S. Y. Mahmud, W. Wang, J. Whitaker, W. Enck, B. Reaves, K. Singh, and T. Xie. PolicyLint: investigating internal privacy policy contradictions on Google Play. In *Proc. 28th USENIX Security Symposium*, 2019.
- [12] H. Cui, R. Trimnanda, A. Markopoulou, and S. Jordan. PoliGraph: automated privacy policy analysis using knowledge graphs. In *Proc. 32nd USENIX Security Symposium*, 2023.
- [13] S. Zimmeck, P. Story, D. Smullen, A. Ravichander, Z. Wang, J. Reidenberg, N. C. Russell, and N. Sadeh. MAPS: scaling privacy compliance analysis to a million apps. *Proceedings on Privacy Enhancing Technologies (PoPETs)*, 2019(3):66-86, 2019.
- [14] M. Lippi, P. Pałka, G. Contissa, F. Lagioia, H.-W. Micklitz, G. Sartor, and P. Torroni. CLAUDETTE: an automated detector of potentially unfair clauses in online terms of service. *Artificial Intelligence and Law*, 27(2):117-139, 2019.

- [15] C. Utz, M. Degeling, S. Fahl, F. Schaub, and T. Holz. (Un)informed consent: studying GDPR consent notices in the field. In *Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2019.
- [16] M. Nouwens, I. Lliccardi, M. Veale, D. Karger, and L. Kagal. Dark patterns after the GDPR: scraping consent pop-ups and demonstrating their influence. In *Proc. CHI Conference on Human Factors in Computing Systems*, 2020.
- [17] C. Matte, N. Bielova, and C. Santos. Do cookie banners respect my choice? Measuring legal compliance of banners from IAB Europe’s transparency and consent framework. In *Proc. IEEE Symposium on Security and Privacy (S&P)*, 2020.
- [18] I. Sanchez-Rola, M. Dell’Amico, P. Kotzias, D. Balzarotti, L. Bilge, P.-A. Vervier, and I. Santos. Can I opt out yet? GDPR and the global illusion of cookie control. In *Proc. ACM Asia Conference on Computer and Communications Security (AsiaCCS)*, 2019.
- [19] T. Libert. An automated approach to auditing disclosure of third-party data collection in website privacy policies. In *Proc. The Web Conference (WWW)*, 2018.
- [20] J. Jueckstock, S. Sarker, P. Snyder, A. Beggs, P. Papadopoulos, M. Varvello, B. Livshits, and A. Kapravelos. Towards realistic and reproducible web crawl measurement. In *Proc. The Web Conference (WWW)*, 2021.
- [21] U. Iqbal, S. Englehardt, and Z. Shafiq. Fingerprinting the fingerprinters: learning to detect browser fingerprinting behaviors. In *Proc. IEEE Symposium on Security and Privacy (S&P)*, 2021.
- [22] Terms of Service; Didn’t Read. <https://tosdr.org>.
- [23] Electronic Frontier Foundation. TOSBack: the terms-of-service tracker. <https://tosback.org>.
- [24] Internet Archive. The Wayback Machine. <https://web.archive.org>.
- [25] Common Crawl Foundation. The Common Crawl corpus. <https://commoncrawl.org>.
- [26] European Parliament and Council. Regulation (EU) 2016/679 (General Data Protection Regulation), Articles 12 to 14. *Official Journal of the European Union*, L119, 2016.
- [27] European Parliament and Council. Regulation (EU) 2022/2065 (Digital Services Act), Article 14. *Official Journal of the European Union*, L277, 2022.
- [28] Open Worldwide Application Security Project. OWASP Top 10:2021, A10 Server-Side Request Forgery. <https://owasp.org/Top10/>